

### EXERCICE 1 (TRI PAR SÉLECTION)

En ne comptabilisant que **les comparaisons**, trouver et analyser en fonction de **n** l'équation de la complexité en temps de l'**algorithme itératif (puis récursif)** suivant :

```
procédure itTriSelection(n, tab) // Permutation tab[i] avec tab[j]
P.F n : Entier (E) Procédure permutation(i, j, tab)
    Tab : tableau (E/S) P.F i, j : Entier (E)
Début Tab : tableau (E/S)
    Pour i variant de 1 à (n-1) faire
        iMin ← indMin(i, n, tab)
        Si i ≠ iMin alors
            permutation(i, iMin, tab)
        Fsi
    Fpour
Fin
Procédure reTriSelection_V1(n, tab)
P.F n : Entier (E)
    tab : Tableau (E/S)
Début // On effectue le tri si le tableau est composé au moins de 2 élt
    Si n > 1 alors
        // Extraction de l'indice du Maximum dans tab[1..n]
        iMax ← IndiceMax(1, n, tab)
        Si i ≠ iMax alors // On effectue la Permutation
            permutation(iMax, n, tab)
        Fsi
        //appel récursif
        reTriSelection_V1(n-1, tab)
    Finsi
Fin
```

### EXERCICE 2 (TRI PAR INSERTION)

```
Procédure itTriInsertion(n, tab)
Paramètres formels :
    n : entier (E)
    tab : tableau de TElement(Entier Par Exemple) (E/S)
Début
    Pour j variant de 2 à n Faire
        // On cherche la place d'insertion de la carte
        carte ← tab[j]
        i ← j-1
        Tantque (i>0) et (carte < tab[i])faire
            Tab[i+1] ← tab[i]
            i ← i - 1
        Fintantque
        // On effectue l'insertion
        Tab[i+1] ← carte
    Finpour
Fin
```

- 1) En ne comptabilisant que **les comparaisons**, trouver et analyser en fonction de **n** l'équation de la complexité en temps de l'**algorithme itératif** ci-dessus.
- 2) Ecrire un algorithme **itératif** qui effectue l'insertion d'un élément « carte » dans un tableau « tab » trié composé de n éléments.
- 3) Ecrire un algorithme **récursif** qui effectue l'insertion d'un élément « carte » dans un tableau « tab » trié composé de n éléments.
- 4) En faisant appel à l'algorithme d'insertion ci-dessus (Question 2 ou 3) ; réécrire une version itérative de l'algorithme de tri par insertion.
- 5) En faisant appel à l'algorithme d'insertion ci-dessus (Question 2 ou 3) ; écrire une version récursive de l'algorithme de tri par insertion.

## TP

- 1) Implémenter en langage C les algorithmes de tris suivants :
  - Tri Par Sélection itératif puis récursif.
  - Tri Par Insertion itératif puis récursif.
  - Tri Fusion
  - Tri Rapide
- 2) Tester votre programme sur les tableaux d'entiers suivants :
  - a) Génération aléatoire d'un tableau de n éléments dont les éléments sont compris entre 100 et 200.
  - b) Génération d'un tableau de n éléments à partir d'un fichier texte en entrée dont la structure est :  
NNb (nombre de nombre à trier)  
VNi (valeur de chaque nombre).
- 3) Étudier graphiquement le temps d'exécution des différents algorithmes de tris.